

CSC 4/583: Programming Project

Adversarial Prompt Discovery in Large Language Models

Graduate Students (200 pts)/Undergraduate Students (250 points)

Due before 11:59 P.M., May 6th

For this project you must submit:

- A repository containing all the source code.
- A project report in PDF format that must contain at least:
 - Instructions on how to compile (if needed) and run the code.
 - Description of the code and methodology.
 - Results for all questions that required programming.
 - Answers for all questions that do not require programming.

Answers are always graded by inspecting both code and documentation. Missing code yields no credit. Missing documentation yields partial credit (if code exists and produces correct results).

The project is worth 200 points of your final grade, with an additional 50 bonus points available for undergraduate students who complete it.

Background

In the current era of frontier models, we’ve grown accustomed to conversational AI systems that have undergone extensive supervised fine-tuning (SFT) and preference optimization (RLHF/DPO). Modern systems like ChatGPT, Claude, and Gemini follow instructions naturally: want something? Just ask the model.

Today, we’ll humble ourselves and travel back to an era where these alignment techniques weren’t in widespread use. We’ll be working with **GPT-2 base**, a pure next-token prediction model trained only on raw internet text. No instruction tuning, no safety training and no conversational polish.

Context: Red Teaming and Adversarial Prompting

Red Teaming in AI safety refers to deliberately trying to make language models behave in unintended ways. Red teams help identify vulnerabilities before deployment. Common activities include testing safety guardrails, finding jailbreaks (prompts that bypass restrictions), and discovering harmful outputs.

LLM Jailbreaking involves crafting prompts that cause aligned models like ChatGPT to violate their safety training. Examples include role-playing prompts (e.g., “DAN - Do Anything Now”), token smuggling (encoding requests in Base64), and context manipulation (pretending harmful content is for a movie script). See <https://www.jailbreakchat.com/> and <https://llmsecurity.net/> for examples.

Our Focus: In this project, we would like to explore the following question: *Can we make a base language model generate tokens it has never seen during training?* This relates to understanding model capabilities, compositional generalization, and prompt engineering fundamentals.

The Challenge

The smallest version of GPT-2, GPT-2 base (124M parameters) was trained on text available before February 2019. It likely has zero knowledge of *The Mandalorian* (premiered November 2019). Yet through careful prompt engineering, it may be possible to elicit character names from this show. Why might this work? GPT-2 uses Byte-Pair Encoding (BPE) — unknown words split into subword units that do exist in the vocabulary. Even if “grogu” was never in training, “gro” and “gu” might appear separately, and certain prompts might increase their co-occurrence probability.

Your task: Find prompts (≤ 10 tokens) that cause GPT-2 to generate these character names:

```
targets = ["grogu", "mando", "kuiil", "peli", "fennec"]
```

Constraints: Prompts must (1) NOT contain the target name, (2) be ≤ 10 tokens when tokenized, and (3) cause GPT-2 to generate the target in its output.

Questions

1) (40 pts) Core Implementation — Manual Prompt Engineering and Evaluation

Before implementing automated search, develop intuition through manual exploration and build the evaluation infrastructure.

Part A: Setup and Manual Exploration (25 pts)

Load GPT-2 and implement basic generation. Verify that simple prompts (e.g., “Star Wars”, “baby yoda”, “bounty hunter”, “din djarin”) do NOT generate the targets. Then systematically explore at least **5 different manual prompt strategies**, such as:

- Substring/prefix prompts: “gro”, “gr ogu”
- Phonetic approaches: “Words that rhyme with vogue”
- Character-level patterns: “g-r-o-g-u spells”
- Acronyms: “The name GROGU stands for”
- Context priming with thematic elements (avoiding direct references)

For EACH approach: document the exact prompt, show GPT-2’s output, explain your hypothesis, and analyze why it worked/failed.

Part B: Test Harness (15 pts)

Implement a validation function:

```
def test_case(model, tokenizer, prompt, keyword, max_tokens=10):  
    """  
    Test if prompt satisfies constraints and generates target.  
  
    Returns: (passed: bool, message: str)  
    """
```

Your implementation must check three conditions: (1) keyword not in prompt (case-insensitive), (2) prompt length $\leq$ max_tokens when tokenized, and (3) keyword appears in generated output.

Note: You might need to handle the issue of tokenization invertibility. During your search, you will most likely work with token IDs directly. However, the final test will validate the *decoded string*. This can cause subtle failures:

```
# During your search:
prompt_ids = [tok1, tok2, ..., tok10] # Exactly 10 tokens
# Optimization succeeds, generates target!

# At test time:
prompt_string = tokenizer.decode(prompt_ids) # Convert to string
test_length = len(tokenizer.encode(prompt_string)) # Re-tokenize
# test_length might be 12! TEST FAILS.
```

Why does this happen? Tokenizers are NOT invertible — the same text can tokenize differently depending on context:

```
tokens = [220, 31373] # [" ", "hello"]
text = tokenizer.decode(tokens) # " hello"
new_tokens = tokenizer.encode(text) # [23748] - single token!
# Original: 2 tokens, Re-encoded: 1 token
```

Solution: During your search, validate candidates using the *re-tokenized* length:

```
# Don't just check: len(prompt_ids) <= 10
# Instead:
decoded = tokenizer.decode(prompt_ids)
actual_length = len(tokenizer.encode(decoded))
if actual_length > max_tokens:
    skip_this_candidate() # Will fail the test!
```

This ensures prompts that work during search will also pass the final test. Ignoring this will cause mysterious failures where your optimization “succeeds” but the test rejects the prompt.

Report success rate, insights from manual exploration, and examples of discovered prompts (if any).

2) (40 pts) Coding with LLMs

For this project you *must* use large language models (LLMs) to help you with coding! Answer the following questions in your report:

- **What LLMs did you use and how?** Which LLM(s) (e.g., ChatGPT, Claude, Gemini, other)? Did you use specialized prompting techniques or “skills” (e.g., <https://github.com/forrestchang/andrej-karpathy-skills>)?
- **Development workflow:** Did you “vibe code” the entire project from scratch with LLM assistance? Did you architect the project yourself and ask the LLM to write specific parts? Did you use the LLM for debugging and hints (like a better Stack Overflow)? Provide specific examples of prompts you gave the LLM and show before/after code snippets.

- **Critical reflection:** What worked and what did not? Which tasks were LLMs particularly good at (e.g., boilerplate, documentation, debugging)? Where did LLMs struggle or mislead you (e.g., incorrect algorithms, hallucinated functions)? How many iterations did complex components require? Would you have completed this faster without LLM assistance? What skills did you develop, and what traditional skills weren't practiced?

Provide concrete examples (screenshots or transcripts of LLM conversations) and quantitative estimates (e.g., “~60% of code was LLM-generated initially, but I rewrote ~30% after debugging”).

3) (70 pts) Automated Prompt Search

Implement a **systematic search algorithm** to find adversarial prompts automatically. This is the core technical contribution of your project.

Objective: Given a target word, automatically discover a valid prompt that causes GPT-2 to generate it.

Approach: You may use any search strategy. We encourage creativity and exploration. Figure 1 illustrates three example approaches at different levels of sophistication. Some possibilities include:

- **Continuous relaxation methods:** Instead of searching over discrete tokens, work in continuous embedding space and project back to vocabulary tokens. You might optimize embeddings directly or use gradients to guide the search (see Gradient Guided approach in Figure 1).
- **Embedding similarity search:** Use vector similarity (cosine distance, L2 distance) in embedding space to find tokens that are “close” to generating the target. You could build candidate prompts by finding tokens whose embeddings have desirable properties, similar to vector database nearest-neighbor search.
- **Discrete search algorithms:** Apply classic AI search methods (beam search, genetic algorithms, simulated annealing, etc.) directly on token sequences, using an objective function to score candidates (see Greedy Search in Figure 1).
- **Hybrid approaches:** Combine continuous and discrete techniques. For example, use continuous methods to identify promising directions, then search discretely in those regions, or use gradients/similarities to score candidates in a discrete search.
- **Corpus-based methods:** Leverage external text data, linguistic patterns, or template structures to generate candidate prompts systematically.
- **Novel methods:** We encourage creative approaches beyond these categories. Justify your design and explain why you believe it will be effective.

Requirements:

Your implementation must:

1. Be fully **automated** (no manual prompt crafting in the main loop)
2. **Optimize** an objective function (e.g., $P(\text{target} \mid \text{prompt})$)
3. **Validate** against all constraints during search (token length, no target in prompt, tokenization stability)
4. **Terminate** after reasonable compute (max iterations or time budget)
5. Be **reproducible** (set random seeds, document hyperparameters)

Evaluation:

Report results in a table:

Target	Success	Prompts Tested	Time (min)	Best Prompt
-----	-----	-----	-----	-----
grogu	Y	2,341	12.3	"..."
mando	Y	891	5.1	"..."
kuiil	N	5,000	28.7	N/A
pelii	Y	1,523	8.9	"..."
fennec	Y	678	3.2	"..."

Compare your method against:

- **Random search baseline:** Generate random valid prompts and test each (see Figure 1, approach 1)
- **Your manual prompts:** From Question 1

Report success rates, efficiency (prompts tested per success), and examples of discovered prompts.

Analysis:

- **Why this method?** Justify your design decisions and discuss alternatives considered. How does your approach compare to the strategies shown in Figure 1?
- **Surprising discoveries:** Did your search find prompts you never would have tried manually? Show examples of unintuitive but successful prompts.
- **Patterns:** Do successful prompts share common characteristics across targets?
- **Efficiency analysis:** How does your method compare to the baselines in terms of prompts evaluated per success? What is the computational cost?

- **Future improvements:** If you had more time/compute, what would you try? Could you combine multiple approaches from Figure 1?

Requirements:

Your implementation must:

1. Be fully **automated**
2. **Validate** against all constraints during search (token length, no target keyword in prompt,)
3. **Terminate** after reasonable compute (max iterations or time budget)
4. Be **reproducible** (set random seeds, document hyperparameters)

Evaluation:

Report results in a table:

Target	Success	Prompts Tested	Time (min)	Best Prompt
-----	-----	-----	-----	-----
grogu	Y	2,341	12.3	"..."
mando	Y	891	5.1	"..."
kuiil	N	5,000	28.7	N/A
...				

Compare your method against:

- **Your manual prompts:** From Question 1

Report success rates, efficiency (prompts tested per success), and examples of discovered prompts.

Analysis:

- **Why this method?** Justify your design decisions and discuss alternatives considered.
- **Surprising discoveries:** Did your search find prompts you never would have tried manually? Show examples of unintuitive but successful prompts.
- **Patterns:** Do successful prompts share common characteristics across targets?
- **Future improvements:** If you had more time/compute, what would you try?

4) (50 pts) Error Analysis

Perform a thorough analysis of your results.

Outcome Categorization: For each target, classify as: (1) Complete Success (valid prompt found, >80% reproducibility), (2) Partial Success (prompt found but inconsistent), (3) Near Miss (generates similar output like “grog” instead of “grogu”), or (4) Complete Failure.

Pattern Identification: Which targets were easier/harder? Analyze based on:

- Tokenization structure (use `tokenizer.encode()` to examine how targets split into subwords)
- Phonetic properties (common vs unusual sound patterns)
- Linguistic overlap with English vocabulary

Constraint Violations: How many candidates violated each constraint? Why do these constraints matter for scientific validity? What would constitute “cheating”?

Provide specific examples throughout with prompts, outputs, and quantitative statistics.

Technical Specifications

Setup

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

model = AutoModelForCausalLM.from_pretrained("gpt2").eval()
tokenizer = AutoTokenizer.from_pretrained("gpt2")

MAX_PROMPT_TOKENS = 10
GENERATION_LENGTH = 32
targets = ["grogu", "mando", "kuiil", "peli", "fennec"]
```

Generation Function

```
def generate(model, tokenizer, prompt, max_new_tokens=32):
    input_ids = tokenizer.encode(prompt, return_tensors="pt")
    with torch.no_grad():
        output_ids = model.generate(
            input_ids,
            max_new_tokens=max_new_tokens,
            do_sample=False # Deterministic
        )[0]
    return tokenizer.decode(output_ids)
```

Grading Scheme

Component	Points	Description
Core Implementation	40	Manual exploration (25), Test harness (15)
LLM Usage	40	Tools and workflow (20), Critical reflection (20)
Automated Search	70	Implementation (30), Experiments (25), Analysis (15)
Error Analysis	50	Categorization (15), Patterns (20), Analysis (15)
TOTAL	200	

Resources

- **GPT-2 Paper:** Radford et al., “Language Models are Unsupervised Multitask Learners” (2019)
- **Transformers Docs:** <https://huggingface.co/docs/transformers>
- **Red Teaming Context:** <https://www.jailbreakchat.com/>, <https://llmsecurity.net/>
- **LLM-Assisted Coding:** <https://github.com/forrestchang/andrej-karpathy-skills>
- **Search Algorithms:** Russell & Norvig, “Artificial Intelligence: A Modern Approach”

Submission Checklist

- ☐ GitHub repository with all code
- ☐ README with setup/run instructions
- ☐ PDF report addressing all questions
- ☐ Code is documented and reproducible
- ☐ Results include tables and examples
- ☐ LLM usage thoroughly documented
- ☐ Citations for external resources

Note: This grading scheme will be used for any custom project approved by the instructor. That is, any alternative project must include: (1) Core implementation (40 pts), (2) LLM usage documentation (40 pts), (3) Main technical contribution (70 pts), and (4) Error analysis (50 pts).

PROMPT SEARCH FOR TARGET GENERATION

Target: “grogu”

1. RANDOM SEARCH

Try random prompts

‘‘hello world’’ $\rightarrow \times$ no “grogu”
‘‘baby alien’’ $\rightarrow \times$ no “grogu”
‘‘gro gu xyz’’ $\rightarrow \checkmark$ generates “grogu”!

2. GREEDY SEARCH

Change one token at a time

Score = $-\log P(\text{grogu} \mid \text{prompt})$
Prompt: [the] [small] [baby]
 $\downarrow$ try replacements
[tiny] [small] [baby] $\rightarrow$ score: 8.2
[green] [small] [baby] $\rightarrow$ score: 7.1 $\leftarrow$ best!
[big] [small] [baby] $\rightarrow$ score: 9.4
Next iteration: [green] [?] [baby]
 $\downarrow$ optimize next position

3. GRADIENT GUIDED

Use gradients to find promising tokens

Position 1: compute $\partial \text{loss} / \partial \text{token}$
Gradient = [-0.8, -0.3, 0.1, -0.9, ...]
 tok₀ tok₁ tok₂ tok₃
 $\uparrow$
 Most negative = tok₃ (best!)
 $\rightarrow$ Select top-k tokens per position
 Evaluate actual loss for each
 Apply best replacement

Figure 1: Three prompt search strategies for eliciting target generation. Random search (baseline) tries prompts randomly. Greedy search iteratively improves one token at a time based on scoring. Gradient-guided methods use calculus to efficiently identify promising token replacements.